FRIEDRICH-SCHILLER-
UNIVERSITÄT
JENA

General Training for Programming Competitions & Coding Interviews

November 6, 2024

Linked Lists, Stacks, Queues, Binary Trees and Tries

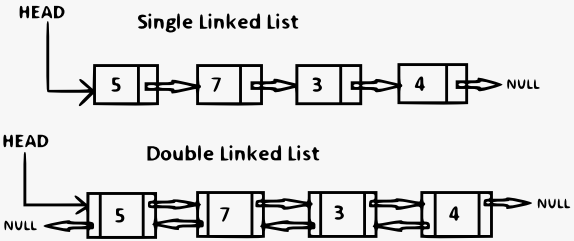
Markus Fischer, Paul G. Rump, Christoph Staudt

Faculty of Mathematics and Computer Science

Linked Lists

Linked Lists Basics

- A **(singly) linked list** is a data structure that contains a sequence of nodes.
- Each node includes an object and a reference to the next node (if any).
- The first node is referred to as the **head** and the last node is referred to as the **tail**.
- In a **doubly linked list**, each node also has a link to its predecessor.



Operations on linked lists

- **Inserting** or **deleting** nodes locally has $\mathcal{O}(1)$ time complexity.
- **Searching** for an element takes $\mathcal{O}(n)$ time.
- Unlike an array, a linked list does not provide **constant time access** to a particular "index" within the list.
- You should have "good reasons" to use a linked list instead of an array in your code.

Delete a Given Node From a Singly Linked List

Given a node in a singly linked list, **deleting it** in $\mathcal{O}(1)$ time appears impossible because its predecessor's next field has to be updated. However, there is a trick that can achieve this if the input node is guaranteed not to be the tail node.

Delete a Given Node From a Singly Linked List

Given a node in a singly linked list, **deleting it** in $\mathcal{O}(1)$ time appears impossible because its predecessor's next field has to be updated. However, there is a trick that can achieve this if the input node is guaranteed not to be the tail node.

delete a node without knowing its predecessor

```
1 // assume node_to_delete is not tail
2 auto successor = node_to_delete->next;
3 // copy (or move) data from successor
4 node_to_delete->data = successor->data;
5 // copy next node from successor
6 node_to_delete->next = successor->next;
7 // delete successor completely from memory
8 delete successor;
```

If we copy the **value part of the next node** to the current node, and then **delete the next node**, we have effectively deleted the current node. The time complexity is $\mathcal{O}(1)$.

The Runner Technique

The **runner** (or second pointer) **technique** is used in many linked list problems.

- Idea: Use two pointers that traverse the list simultaneously.
- The fast pointer could be ahead with a fixed offset, or could use multiple steps each time the slow pointer moves one.
- (Compare this to the techniques we used to find & delete elements from a string/array.)

The Runner Technique

The **runner** (or second pointer) **technique** is used in many linked list problems.

- Idea: Use two pointers that traverse the list simultaneously.
- The fast pointer could be ahead with a fixed offset, or could use multiple steps each time the slow pointer moves one.
- (Compare this to the techniques we used to find & delete elements from a string/array.)

Task: Find the midpoint of a linked list in a single iteration.

The Runner Technique

The **runner** (or second pointer) **technique** is used in many linked list problems.

- Idea: Use two pointers that traverse the list simultaneously.
- The fast pointer could be ahead with a fixed offset, or could use multiple steps each time the slow pointer moves one.
- (Compare this to the techniques we used to find & delete elements from a string/array.)

Task: Find the midpoint of a linked list in a single iteration.

Solution:

- One pointer moves twice as quickly as the other.
- When the fast pointer reaches the end, the slow pointer is in the middle.
- In practice: Handle lists of even length safely.

Test for Cyclicity

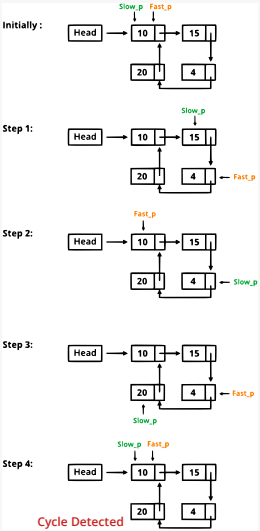
Write a program that takes the head of a singly linked list and finds out whether there is a **cycle** in the list.
(Requirements: $\mathcal{O}(1)$ space, $\mathcal{O}(n)$ time)

Test for Cyclicity

Write a program that takes the head of a singly linked list and finds out whether there is a **cycle** in the list.
(Requirements: $\mathcal{O}(1)$ space, $\mathcal{O}(n)$ time)

has_cycle.cpp

```cpp
bool has_cycle(Node *node) {
  Node *slow = node;
  Node *fast = node;
  while (fast && fast->next) {
    // advance slow by one
    slow = slow->next;
    // advance fast by two
    fast = fast->next->next;
    if (slow == fast) {
      return true;
    }
  }
  return false;
}
```



Test for Cyclicity

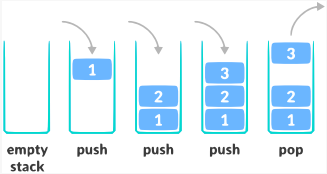
This works, because when the the slow pointer enters the cycle it is in **front of the fast pointer**. Since the fast pointer moves twice as fast, the distance between them is reduced by 1 in each step, until they meet.
Therefore they meet while the slow pointer traverses the cycle for the first time.

If the fast pointer reaches `null`, there is no cycle!

Stacks & Queues

Stacks Basics

- A stack uses **LIFO** (last-in, first-out) ordering. That is, as in a stack of dinner plates, the item that was **added last** is **removed first**.



- A **stack can be implemented** with a **linked list** or a dynamically **resizable array**.
- Simple **parsing problems** can often be implemented with a stack.
  (Learn to recognize problems when the stack LIFO property is applicable.)
- A stack can also be used to **implement a recursive algorithm iteratively**.
- **Consider using the C++ stack class** instead of implementing your own stack class.
  https://www.geeksforgeeks.org/stack-in-cpp-stl/

A `std::vector` Can Be Also Used as a Stack

vector_stack.cpp

```cpp
// A vector can be used as a stack.
// This (unnecessary) wrapper demonstrates how.
template <typename T> class Stack {
private:
  std::vector<T> data;

public:
  void push(T item) { this->data.push_back(item); }
  bool empty() { return this->data.empty(); }
  T top() { return this->data.back(); }
  void pop() { this->data.pop_back(); }
};
```

A `std::vector` semantically **offers stack functionality**. Using a vector directly as a stack **can speed up code**.

Stack in Python

In python you can also use a **list** as an easy stack implementation.

```
# initialize stack as a list:
stack = []
# push an element to the end of the stack:
stack.append(5)
# check if stack is empty:
len(stack) == 0
# get topmost (i.e. last) element:
stack.pop(-1)
```

However, this is **not the fastest way in python** if the stack becomes large. In the case of large stacks, better use **Collections.deque** or **queue.LifoQueue** (see https://www.geeksforgeeks.org/stack-in-python/).

Test a String Over {}()[] for Well-Formedness

A string over the characters {}()[] is said to be **well-formed if** the **different types of brackets match in the correct order**. For example, ([]){()} is well-formed. However, {) and [()[]{()() are not well-formed. Write a program that tests if a string made up of the characters {}()[] is well-formed.

Test a String Over {}()[] for Well-Formedness

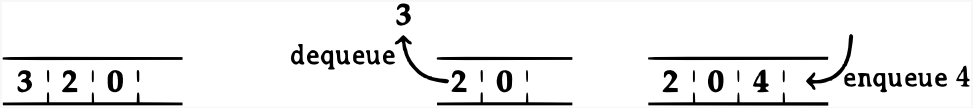
A string over the characters {}()[] is said to be **well-formed if** the **different types of brackets match in the correct order**. For example, ([]){()} is well-formed. However, {) and [()[]{()() are not well-formed. Write a program that tests if a string made up of the characters {}()[] is well-formed.

is_well_formed.cpp

```
1
2  bool is_well_formed(const string &s) {
3    stack<char> chars;
4    for (const char c : s) { // iterate over all characters in s
5      if (c == '(' || c == '{' || c == '[') {
6        chars.push(c);
7      } else {
8        // missing corresponding left bracket
9        if (chars.empty()) { return false; }
10       if ((c == ')' && chars.top() != '(') ||
11           (c == '}' && chars.top() != '{') ||
12           (c == ']' && chars.top() != '[')) {
13         return false;
14       }
15       chars.pop();
16     }
17   }
18   // if stack not empty we have unmatched left brackets
19   return chars.empty();
```

Queues Basics

- Add (**enqueue**) and remove (**dequeue**) elements in **FIFO** (first-in, first-out) order.

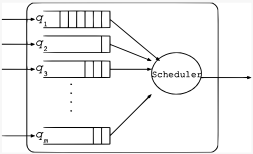


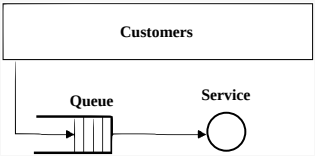
- A **queue can be implemented efficiently** using a **linked list** or a (dynamically resizable circular) **array**.

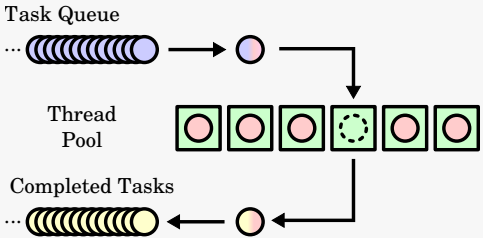
Queues in practice

queue_with_list.cpp

```cpp
// simple queue implementation with a list
template <typename T> class Queue {
private:
  list<T> data;

public: // a std::list can be used as a queue
  void enqueue(T x) { data.emplace_back(x); }

  T dequeue() {
    const T val = data.front();
    data.pop_front();
    return val;
  }

  // convenience functions
  size_t size() { return data.size(); }
  bool empty() { return data.empty(); }
```







Binary Trees & Recursion

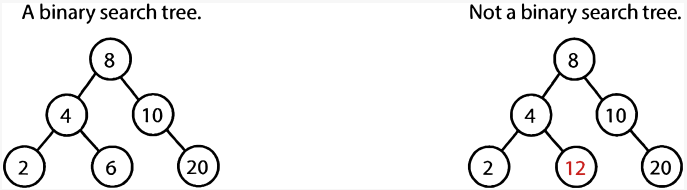
Binary Trees Basics

A tree is a **type of graph**. (We will cover general graphs later.)

- Each tree has a **root node**.
- The root and each other node has zero or more **child nodes**. Nodes without children are also called **leaf nodes**.
- The tree **must not contain cycles**. The nodes may or may not be in a particular order, they could have any data type as values, and they may or may not **have links back to their parent nodes**.
- The **height** of a tree is the length of the longest path from the root to a leaf.

Binary Trees Basics

A binary tree is a tree in which each node **has up to two children**. A binary search tree imposes the condition that, for each node, its **left descendents are less than or equal to** the current node, **which is less than the right descendents**.

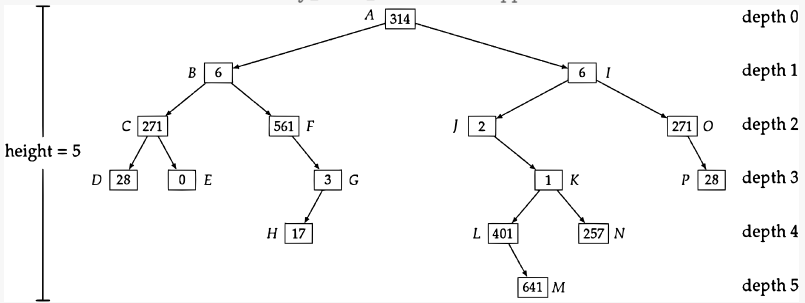


Special kinds of balanced binary search trees are the **Red-Black tree**, and the **AVL tree**.

Binary Tree Traversal

You should be comfortable implementing **in-order**, **post-order**, and **pre-order** traversal.

binary_tree_traversal.cpp



pre-order

```
// A B C D E F G H I J K L M N O P
void pre_order(Binary_Tree_Node *node) {
  if (node) {
    print(node); // process node here
    pre_order(node->left);
    pre_order(node->right);
  }
}
```

in-order

```
// D C E B F H G A J L M K N I O P
void in_order(Binary_Tree_Node *node) {
  if (node) {
    in_order(node->left);
    print(node); // process node here
    in_order(node->right);
  }
}
```

post-order

```
// D E C H G F B M L N K J P O I A
void post_order(Binary_Tree_Node *node) {
  if (node) {
    post_order(node->left);
    post_order(node->right);
    print(node); // process node here
  }
}
```

Binary Tree Traversal Without Recursion

Often it is more convenient to solve **binary tree problems** by traversing the tree **with recursion**. But using a stack, we can implement tree traversals without recursion.

binary_tree_traversal_no_recursion.cpp

pre-order without recursion

```
void pre_order_no_recursion(Binary_Tree_Node *node) {
  stack<Binary_Tree_Node *> s;
  s.push(node);
  while (!s.empty()) {
    node = s.top();
    s.pop();
    if (node != nullptr) {
      print(node); // process node here
      // add left child after right child, since we
      // want to continue with the left child
      s.push(node->right);
      s.push(node->left);
    }
  }
}
```

in-order without recursion

```
void in_order_no_recursion(Binary_Tree_Node *node) {
  stack<Binary_Tree_Node *> s;
  while (!s.empty() || node) {
    if (node) {
      s.push(node);
      node = node->left; // going left
    } else {
      node = s.top(); // going up
      s.pop();
      print(node);        // process node here
      node = node->right; // going right
    }
  }
}
```

Recursion Elimination

Solving problems by recursion is often the intuitive way. However, recursive functions can be bad for the performance of your code:

- All recursive calls require memory.
- Recursive calls are managed by a stack, this can overflow if the recursion is too deep.
- For example, python has a *RecursionError: maximum recursion depth exceeded* error to prevent the stack overflow.

useless_recursion.py

```
def useless_recursion(n):
   if n == 0:
      print ("done.")
      return
   else:
      return useless_recursion(n-1)

if __name__ == "__main__":
   print ("case 990:")
   useless_recursion(990)
   print ("case 1000:")
   useless_recursion(1000)
```

Recursion Elimination

Try to eliminate your recursion, or transform it into a tail-recursion!

Bad recursion:

recursion.cpp

```cpp
int factorial_bad(unsigned int a){
   // compute the factorial a! using a recursive function
   if (a == 0){
      return 1;
   }
   return a * factorial_bad(a-1);
}
```

- There is another operation happening *after* the recursive call!

Good (tail-) recursion:

recursion.cpp

```cpp
int factorial_tail(unsigned int a, unsigned long int ret=1){
   // compute the factorial a! using a tail-recursive function
   if (a < 2){
      return ret;
   }
   return factorial_tail(a-1, ret*a);
}
```

- Recursive call is the *last* operation.
- Can be automatically transformed into while-loop by compiler!

Recursion Elimination

Computing the n-th **Fibbonacci number** is the prime example of recursive algorithms:

Fibbonacci recursive

```
1 unsigned long int fib_rec(unsigned int n){
2    if (n<=2){
3       return 1;
4    }
5    return fib_rec(n-1)+fib_rec(n-2);
6 }
```

There is an obvious non-recursive way to compute the Fibbonacci number by a for-loop. But for the sake of this exercise, let us try to write this function as a tail-recursion.

Recursion Elimination

Computing the *n*-th **Fibbonacci number** as a tail recursion:

Fibbonacci tail-recursive

```
1  unsigned long int fib_tailrec(unsigned int n, unsigned int k=1, unsigned int last=1, unsigned int seclast=1){
2     // n : target fibonacci number
3     // k : current fibonacci number
4     // last, seclast: last and second-to-last numbers
5
6     if (n <= 2){
7        return 1;
8     }
9     if (k+1 == n){
10       return last;
11    }
12    return fib_tailrec(n, k+1, last+seclast, last);
13 }
```

Recursion Elimination

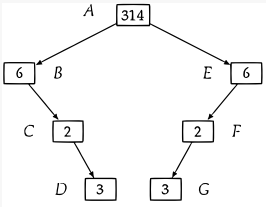
Depending on the optimization flag `-O`, `g++` can detect and eliminate some recursions automatically.

Experiments: compile and run `fibbo.cpp` and `recursion_squares.cpp` with different optimization flags (`-O0`, `-O1`, `-O2`).

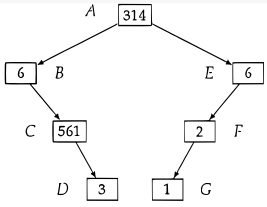
But *better save than sorry*: try to write your code directly without recursion, don't rely on the compiler!

Test if a Binary Tree Is Symmetric

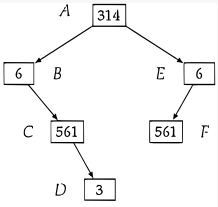
A binary tree is **symmetric** if you can draw a vertical line through the root and then the left sub-tree is the mirror image of the right sub-tree.



A symmetric binary tree.



An asymmetric binary tree.



An asymmetric binary tree.

Test if a Binary Tree Is Symmetric

Write a **program that checks** whether a **binary tree is symmetric**

Test if a Binary Tree Is Symmetric

Write a **program that checks** whether a **binary tree is symmetric**

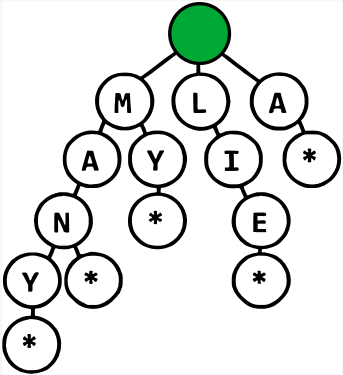
is_symmetric.cpp

```cpp
bool check_symmetric(Binary_Tree_Node *subtree_left, Binary_Tree_Node *subtree_right) {
  if (subtree_left == nullptr && subtree_right == nullptr) {
    return true;
  } else if (subtree_left != nullptr && subtree_right != nullptr) {
    return subtree_left->data == subtree_right->data &&
           check_symmetric(subtree_left->left, subtree_right->right) &&
           check_symmetric(subtree_left->right, subtree_right->left);
  }
  return false; // one subtree is empty, and the other is not
}

bool is_symmetric(Binary_Tree_Node *node) {
  return node == nullptr || check_symmetric(node->left, node->right);
}
```

Prefix Trees ("Tries")

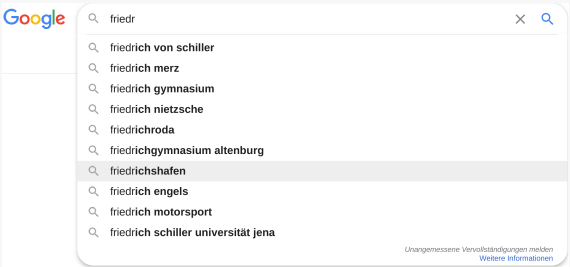
Prefix Trees ("Tries")

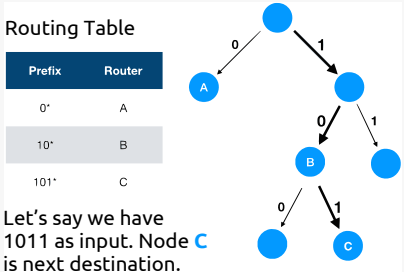
- A trie comes up a lot in interview questions, but algorithm textbooks don't spend much time on this data structure.
- The word **trie** is an infix of the word "re**trie**val".
  - A trie is a variant of an n-ary tree in which **characters are stored at each node**. **Each path** down the tree **may represent a word**.
  - The *** nodes** (sometimes called "null nodes") are often used to **indicate complete words**.

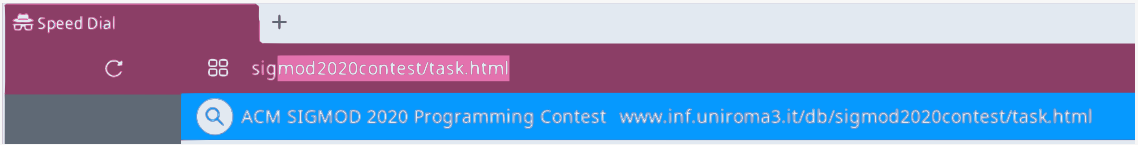


Where Are Tries Used?

Very commonly, tries are used to store entire dictionaries for quick **prefix lookups**. While a hash table can quickly look up whether a string is a valid word, it cannot tell us if a string is a prefix of any valid words. A trie can do this in $\mathcal{O}(K)$ time, where K is the length of the string.

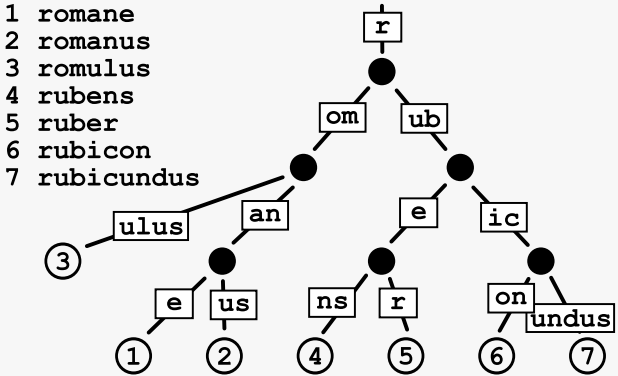






Radix Trees

A **space-optimized** (compressed) trie in which each node that is the only child is merged with its parent is called a **radix tree**.
(Read about **radix trees in databases**: https://db.in.tum.de/~leis/papers/ART.pdf)



Exercises (Upload Solutions to your Repository)

1. **Check if** a **linked list** is a **palindrome**.

   `is_palindrom.cpp` 

2. Write a program to **evaluate arithmetic expressions** in **Reverse Polish Notation**, for example a term like:

   `5 4 +`.

   `eval_rpn.cpp` 

3. **Implement** a **queue** by using **two stacks**.

   `queue_with_stacks.cpp`

4. **Implement** a function to **check if** a **binary tree is** a **binary search tree**.

   `is_binary_search_tree.cpp` 

5. **Find** the **shortest prefix** of a **string** which is **not a prefix of any string in the trie**.

   `shortest_prefix.cpp` 

Exam Questions

- **How** to **delete** a (non-tail) **node** from a **singly linked list without knowing its predecessor** in $\mathcal{O}(1)$ time?
- Explain the **runner technique** using a linked list **example**.
- If you were to **tune** the **code for speed**, **how would you implement a stack**?
- **How** can we **use a** `std::list` in C++ **as a queue**?
- **How** does a **binary search tree** differ from a **binary tree**?
- **Describe** the **three types** of **binary tree traversals**.
- **How** does a **radix tree** differ from a **trie**?